

CLI Command Spec

Local AI Software Team

1. Document Overview

Product Name

Local AI Software Team

CLI Name

Recommended command name:

```
aiteam
```

Alternative command names:

```
specforge  
reqpilot  
teamforge
```

This document uses `aiteam` as the default CLI command.

Purpose

This document defines the CLI commands for Local AI Software Team.

The CLI is the main execution interface of the product. It allows users to initialize projects, start the local web UI, create workflow runs, generate documents, coordinate AI coding agents, run tests, review results, and generate final reports.

The CLI should work without a backend cloud and without user login.

2. CLI Design Principles

2.1 Local-first

All commands should run locally.

The CLI should not require:

- cloud backend
- login

- SaaS account
- online workspace

2.2 Safe by default

The CLI should avoid risky actions unless the user explicitly approves.

Default behavior:

- do not auto-commit
- do not auto-push
- do not auto-merge
- do not update Jira/Slack/GitHub without approval
- do not edit protected files without approval
- do not run dangerous commands

2.3 Workflow-first

The CLI is not just a command wrapper for AI tools.

It should guide the full workflow:

```
raw requirement
→ clarified requirement
→ technical design
→ task breakdown
→ test matrix
→ implementation prompt
→ optional code generation
→ test
→ review
→ traceability
→ final report
```

2.4 Step-by-step or full-run

The CLI should support both:

Full-run mode

```
aiteam run "Thêm API export invoice CSV theo hotelId, date range, status"
```

Step-by-step mode

```
aiteam new "Thêm API export invoice CSV theo hotelId, date range, status"
aiteam spec
aiteam plan
```

```
aiteam code --agent codex
aiteam test
aiteam review
aiteam report
```

2.5 Human-readable output

CLI output should be clear and useful.

Every command should show:

- current step
- output files created
- next suggested command
- errors if any
- final status

3. Global Command Structure

```
aiteam <command> [arguments] [options]
```

Examples:

```
aiteam init
aiteam ui
aiteam doctor
aiteam run "Add password reset by email OTP"
aiteam spec --run run_123
aiteam plan --run run_123
aiteam code --run run_123 --agent codex
aiteam test --run run_123
aiteam review --run run_123
aiteam report --run run_123
```

4. Global Options

These options should be available for most commands.

```
--project <path>
```

Project root path. Defaults to current working directory.

```
--config <path>
```

Path to custom config file. Defaults to `.ai-team/config.json`.

```
--run <runId>
```

Target run ID.

```
--verbose
```

Show detailed logs.

```
--json
```

Output machine-readable JSON.

```
--no-color
```

Disable colored terminal output.

```
--yes
```

Auto-confirm safe prompts. Must not bypass dangerous approval gates unless explicitly allowed by command.

```
--dry-run
```

Show what would happen without executing changes.

```
--help
```

Show command help.

```
--version
```

Show CLI version.

5. Exit Codes

The CLI should use consistent exit codes.

Exit Code	Meaning
0	Success
1	General error
2	Invalid arguments
3	Project not initialized
4	Config error
5	AI CLI not found
6	AI CLI execution failed
7	Git error
8	Test failed
9	Command timeout
10	Dangerous command blocked
11	Protected file modified
12	Missing credential
13	Integration failed
14	Workflow cancelled
15	Approval required

6. Command List

6.1 Project and Environment Commands

```
aiteam init
aiteam ui
aiteam doctor
aiteam config
aiteam status
```

6.2 Workflow Commands

```
aiteam new
aiteam run
```

```
aiteam resume  
aiteam cancel  
aiteam list  
aiteam show
```

6.3 Stage Commands

```
aiteam spec  
aiteam scope  
aiteam analyze  
aiteam plan  
aiteam tasks  
aiteam tests  
aiteam code  
aiteam test  
aiteam review  
aiteam trace  
aiteam report
```

6.4 Artifact Commands

```
aiteam artifacts  
aiteam open  
aiteam export  
aiteam clean
```

6.5 Integration Commands

```
aiteam github  
aiteam jira  
aiteam slack
```

6.6 Prompt and Template Commands

```
aiteam prompts  
aiteam templates
```

7. Command: `aiteam init`

Purpose

Initialize Local AI Software Team in the current project.

This command creates the `.ai-team` directory, default config, prompt templates, local database, and run folder structure.

Syntax

```
aiteam init [options]
```

Options

```
--type <type>
```

Project type.

Supported values:

```
java-spring-boot  
node-nestjs  
react-vite  
generic
```

```
--force
```

Overwrite existing `.ai-team` setup.

```
--no-detect
```

Skip automatic project detection.

```
--output-language <language>
```

Default output language for generated docs.

Supported values:

```
en
vi
bilingual
```

Examples

```
aiteam init
```

```
aiteam init --type java-spring-boot
```

```
aiteam init --type java-spring-boot --output-language bilingual
```

Actions

The command should:

1. Detect current working directory.
2. Check if directory is a Git repository.
3. Create `.ai-team/`.
4. Create `.ai-team/config.json`.
5. Create `.ai-team/database.sqlite`.
6. Create `.ai-team/prompts/`.
7. Copy default prompt templates.
8. Create `.ai-team/runs/`.
9. Detect project type if possible.
10. Suggest build/test commands.
11. Print next steps.

Output

Example output:

```
Local AI Software Team initialized.
```

```
Project: hotel-booking-service
Type: java-spring-boot
Config: .ai-team/config.json
Prompts: .ai-team/prompts
Runs: .ai-team/runs
```

```
Detected:
```

- Build tool: Maven
- Test command: mvn test
- Build command: mvn clean package -DskipTests

Next:

1. Run: aiteam doctor
2. Run: aiteam ui
3. Or start with: aiteam run "your requirement"

Generated Files

```
.ai-team/  
  config.json  
  database.sqlite  
  prompts/  
    ba-agent.md  
    po-agent.md  
    pm-agent.md  
    architect-agent.md  
    developer-agent.md  
    qa-agent.md  
    reviewer-agent.md  
    security-reviewer-agent.md  
    reporter-agent.md  
  runs/
```

Error Cases

Already initialized

Error: Project already initialized.

.ai-team already exists.
Use --force to reinitialize.

Exit code: 4

Not writable

Error: Cannot create .ai-team directory.
Check folder permissions.

Exit code: 1

8. Command: `aiteam ui`

Purpose

Start the local web UI.

The local web UI is used for settings, run history, document viewing, logs, prompt templates, integrations, and approval screens.

Syntax

```
aiteam ui [options]
```

Options

```
--port <port>
```

Port for local server. Default: `4317`.

```
--host <host>
```

Host address. Default: `127.0.0.1`.

```
--open
```

Automatically open browser.

```
--no-open
```

Do not open browser.

Examples

```
aiteam ui
```

```
aiteam ui --port 4321 --open
```

Actions

The command should:

1. Load project config.
2. Start local API server.
3. Serve local web UI.
4. Print local URL.
5. Keep process running until stopped.

Output

```
Local AI Software Team UI is running.
```

```
URL: http://localhost:4317
```

```
Press Ctrl+C to stop.
```

Error Cases

Project not initialized

```
Error: Project not initialized.  
Run: aiteam init
```

Exit code: 3

Port already in use

```
Error: Port 4317 is already in use.  
Try: aiteam ui --port 4321
```

Exit code: 1

9. Command: `aiteam doctor`

Purpose

Check whether the local environment is ready.

Syntax

```
aiteam doctor [options]
```

Options

```
--fix
```

Attempt safe automatic fixes.

```
--json
```

Output JSON.

Examples

```
aiteam doctor
```

```
aiteam doctor --json
```

Checks

The command should check:

- Node.js installed
- Git installed
- current folder is Git repository
- `.ai-team` initialized
- config valid
- SQLite accessible
- Claude Code installed
- Codex CLI installed
- Gemini CLI installed
- Aider installed
- Maven/Gradle/npm depending on project
- GitHub CLI installed if GitHub integration enabled
- Jira token available if Jira integration enabled
- Slack token available if Slack integration enabled

Output

```
AI Team Doctor
```

```
Project
  .ai-team initialized: OK
  Config valid: OK
  SQLite: OK

System
  Node.js: OK
  Git: OK

AI CLI
  Claude Code: OK
  Codex CLI: Not found
  Gemini CLI: OK
  Aider: Not found

Project Tools
  Maven: OK
  Gradle: Not found

Integrations
  GitHub CLI: OK
  Jira token: Missing
  Slack token: Missing

Status: READY_WITH_WARNINGS
```

Exit Behavior

- Exit code `0` if ready.
- Exit code `1` if critical error.
- Exit code `5` if selected default AI CLI is missing.
- Exit code `12` if required credential is missing.

10. Command: `aiteam config`

Purpose

View or modify local configuration.

Syntax

```
aiteam config <subcommand> [options]
```

Subcommands

```
aiteam config get <key>
aiteam config set <key> <value>
aiteam config list
aiteam config validate
aiteam config path
```

Examples

```
aiteam config list
```

```
aiteam config get agents.defaultDeveloper
```

```
aiteam config set agents.defaultDeveloper codex
```

```
aiteam config set commands.unitTest "mvn test"
```

```
aiteam config validate
```

Output

```
agents.defaultDeveloper = codex
```

Error Cases

Unknown key

```
Error: Unknown config key: agents.defaultBackend
```

Exit code: 2

Invalid value

```
Error: Invalid value for agents.defaultDeveloper.
Allowed values: claude, codex, gemini, aider
```

Exit code: 4

11. Command: `aiteam status`

Purpose

Show current project status and latest workflow run status.

Syntax

```
aiteam status [options]
```

Options

```
--run <runId>
```

Show status for a specific run.

```
--json
```

Output JSON.

Examples

```
aiteam status
```

```
aiteam status --run run_20260622_143000
```

Output

```
Project: hotel-booking-service
Type: java-spring-boot
Current branch: feature/invoice-export
```

```
Latest Run:
  ID: run_20260622_143000
  Title: Export invoice CSV
  Mode: docs-only
  Status: REPORT_GENERATED
```

```
Artifacts:
  Requirement: generated
  Design: generated
  Tasks: generated
```

Tests: generated
Report: generated

12. Command: `aiteam new`

Purpose

Create a new workflow run from a raw requirement, but do not execute workflow stages automatically.

Syntax

```
aiteam new <requirement> [options]
```

Arguments

```
<requirement>
```

Raw requirement text.

Options

```
--title <title>
```

Custom run title.

```
--mode <mode>
```

Workflow mode.

Supported values:

```
docs-only  
assisted  
semi-auto  
multi-agent
```

```
--language <language>
```

Input language.

Supported values:

```
auto
en
vi
```

```
--output-language <language>
```

Output language.

Supported values:

```
en
vi
bilingual
```

Examples

```
aiteam new "Thêm chức năng reset password bằng email OTP"
```

```
aiteam new "Add invoice CSV export" --title "Invoice CSV Export" --mode docs-only
```

Actions

The command should:

1. Create run ID.
2. Create run folder.
3. Save `input.md`.
4. Save run metadata.
5. Print next suggested command.

Output

```
New run created.
```

```
Run ID: run_20260622_143000
Title: Reset password by email OTP
Mode: docs-only
```

```
Input saved:
.ai-team/runs/run_20260622_143000/input.md
```

Next:
aiteam spec --run run_20260622_143000

13. Command: aiteam run

Purpose

Run a workflow from raw requirement.

This is the main command for end-to-end execution.

Syntax

```
aiteam run <requirement> [options]
```

Arguments

```
<requirement>
```

Raw requirement text.

Options

```
--mode <mode>
```

Workflow mode.

Supported values:

```
docs-only  
assisted  
semi-auto  
multi-agent
```

Default: `docs-only`.

```
--agent <agent>
```

Selected AI coding agent for implementation.

Supported values:

```
claude  
codex  
gemini  
aider
```

```
--title <title>
```

Custom run title.

```
--max-iterations <number>
```

Maximum fix iterations. Default: .

```
--no-code
```

Skip code generation.

```
--no-test
```

Skip test execution.

```
--no-review
```

Skip review stage.

```
--output-language <language>
```

Output language.

Supported values:

```
en  
vi  
bilingual
```

```
--approve
```

Approve non-risky gates automatically. Must not bypass risky file or external update gates.

```
--dry-run
```

Show workflow plan without executing.

Examples

```
aiteam run "Thêm API export invoice CSV theo hotelId, date range, status"
```

```
aiteam run "Add password reset by email OTP" --mode assisted
```

```
aiteam run "Add password reset by email OTP" --mode semi-auto --agent codex
```

```
aiteam run "Create expense approval system" --mode docs-only --output-  
language bilingual
```

Docs-only Flow

```
new  
→ spec  
→ scope  
→ plan  
→ tasks  
→ tests  
→ report
```

Assisted Flow

```
new  
→ spec  
→ scope  
→ analyze  
→ plan  
→ tasks  
→ tests  
→ implementation prompt  
→ report
```

Semi-auto Flow

```
new  
→ spec
```

- scope
- analyze
- plan
- tasks
- tests
- approval
- code
- test
- review
- trace
- report

Output

```
Run started.

Run ID: run_20260622_143000
Mode: docs-only
Requirement: Add password reset by email OTP

[1/6] Generating requirement spec... OK
[2/6] Defining scope... OK
[3/6] Generating technical plan... OK
[4/6] Generating task breakdown... OK
[5/6] Generating test matrix... OK
[6/6] Generating final report... OK

Run completed.

Artifacts:
- requirement/clarified-requirement.md
- requirement/acceptance-criteria.md
- design/technical-design.md
- tasks/task-breakdown.md
- tests/test-matrix.md
- report/final-report.md

Open in UI:
aiteam ui
```

Error Cases

AI CLI missing

```
Error: Selected agent "codex" is not available.
Run: aiteam doctor
```

Exit code: 5

Approval required

Approval required before code generation.

Run:

```
aiteam code --run run_20260622_143000 --agent codex
```

Exit code: 15

14. Command: `aiteam resume`

Purpose

Resume a paused, failed, or incomplete run.

Syntax

```
aiteam resume <runId> [options]
```

Options

```
--from <stage>
```

Resume from a specific stage.

Supported values:

```
spec  
scope  
analyze  
plan  
tasks  
tests  
code  
test  
review  
trace  
report
```

```
--force
```

Force rerun from selected stage.

Examples

```
aiteam resume run_20260622_143000
```

```
aiteam resume run_20260622_143000 --from review
```

Output

```
Resuming run: run_20260622_143000  
Current status: TEST_FAILED  
Next stage: review  
  
[1/1] Generating review report... OK
```

15. Command: `aiteam cancel`

Purpose

Cancel a running or paused workflow.

Syntax

```
aiteam cancel <runId> [options]
```

Options

```
--reason <reason>
```

Cancellation reason.

Examples

```
aiteam cancel run_20260622_143000
```

```
aiteam cancel run_20260622_143000 --reason "Requirement changed"
```

Output

Run cancelled.

Run ID: run_20260622_143000

Reason: Requirement changed

16. Command: aiteam list

Purpose

List workflow runs.

Syntax

```
aiteam list [options]
```

Options

```
--limit <number>
```

Number of runs to show. Default: 20.

```
--status <status>
```

Filter by status.

```
--mode <mode>
```

Filter by mode.

```
--json
```

Output JSON.

Examples

```
aiteam list
```

```
aiteam list --status REPORT_GENERATED
```

Output

Recent Runs

ID	Status	Mode	Title
run_20260622_143000	REPORT_GENERATED	docs-only	Export invoice CSV
run_20260622_151200	TEST_FAILED	semi-auto	Reset password OTP
run_20260621_180500 approval	PLAN_GENERATED	assisted	Booking cancellation

17. Command: `aiteam show`

Purpose

Show details of a specific run.

Syntax

```
aiteam show <runId> [options]
```

Options

```
--artifacts
```

Show generated artifacts.

```
--logs
```

Show log paths.

```
--json
```

Output JSON.

Examples

```
aiteam show run_20260622_143000
```

```
aiteam show run_20260622_143000 --artifacts
```

Output

```
Run: run_20260622_143000  
Title: Export invoice CSV  
Mode: docs-only  
Status: REPORT_GENERATED  
Created: 2026-06-22 14:30
```

Artifacts:

- input.md
- requirement/clarified-requirement.md
- requirement/acceptance-criteria.md
- design/technical-design.md
- tasks/task-breakdown.md
- tests/test-matrix.md
- report/final-report.md

18. Command: `aiteam spec`

Purpose

Generate clarified requirement, business rules, assumptions, open questions, and acceptance criteria.

Syntax

```
aiteam spec [options]
```

Options

```
--run <runId>
```

Target run. Defaults to latest run.

```
--agent <agent>
```

Agent used for generation.

Supported values:

```
claude  
codex  
gemini  
aider
```

```
--regenerate
```

Regenerate existing spec.

```
--output-language <language>
```

Output language.

```
--with-questions
```

Emphasize clarification questions.

```
--assume
```

Allow agent to make assumptions and continue.

Examples

```
aiteam spec
```

```
aiteam spec --run run_20260622_143000
```

```
aiteam spec --run run_20260622_143000 --output-language bilingual
```

Generated Artifacts

```
requirement/  
clarified-requirement.md
```

```
business-rules.md
acceptance-criteria.md
open-questions.md
assumptions.md
```

Output

Requirement spec generated.

Artifacts:

- requirement/clarified-requirement.md
- requirement/business-rules.md
- requirement/acceptance-criteria.md
- requirement/open-questions.md
- requirement/assumptions.md

Next:

aiteam scope --run run_20260622_143000

19. Command: aiteam scope

Purpose

Generate scope definition, MVP scope, out-of-scope list, priorities, and success criteria.

Syntax

```
aiteam scope [options]
```

Options

```
--run <runId>
```

Target run.

```
--regenerate
```

Regenerate existing scope.

```
--strict
```

Make scope smaller and more MVP-focused.

Examples

```
aiteam scope --run run_20260622_143000
```

```
aiteam scope --run run_20260622_143000 --strict
```

Generated Artifacts

```
scope/  
  scope-definition.md  
  out-of-scope.md  
  success-criteria.md
```

Output

Scope generated.

Artifacts:

- scope/scope-definition.md
- scope/out-of-scope.md
- scope/success-criteria.md

Next:

```
aiteam analyze --run run_20260622_143000
```

20. Command: `aiteam analyze`

Purpose

Analyze repository context.

This is especially useful for brownfield projects.

Syntax

```
aiteam analyze [options]
```

Options

```
--run <runId>
```

Target run.

```
--path <path>
```

Repository path. Defaults to project root.

```
--deep
```

Perform deeper analysis.

```
--include <glob>
```

Include file pattern.

```
--exclude <glob>
```

Exclude file pattern.

```
--no-ai
```

Use only deterministic repository detection without AI summary.

Examples

```
aiteam analyze --run run_20260622_143000
```

```
aiteam analyze --run run_20260622_143000 --deep
```

```
aiteam analyze --include "src/main/java/**" --exclude "target/**"
```

Generated Artifacts

```
design/  
  repository-analysis.md  
  project-summary.json
```

```
affected-modules.md
existing-patterns.md
```

Output

Repository analysis completed.

Detected:

- Language: Java
- Framework: Spring Boot
- Build tool: Maven
- Test framework: JUnit
- Migration: Flyway

Artifacts:

- design/repository-analysis.md
- design/project-summary.json
- design/affected-modules.md
- design/existing-patterns.md

Next:

```
aiteam plan --run run_20260622_143000
```

21. Command: `aiteam plan`

Purpose

Generate technical design and implementation plan.

Syntax

```
aiteam plan [options]
```

Options

```
--run <runId>
```

Target run.

```
--regenerate
```

Regenerate plan.

```
--level <level>
```

Planning detail level.

Supported values:

```
simple
standard
detailed
```

Default: `standard`.

```
--stack <stack>
```

Specify stack manually.

Example:

```
java-spring-boot
node-nestjs
react-vite
```

Examples

```
aiteam plan --run run_20260622_143000
```

```
aiteam plan --run run_20260622_143000 --level detailed
```

Generated Artifacts

```
design/
  technical-design.md
  api-design.md
  db-design.md
  service-flow.md
  risk-analysis.md
  implementation-plan.md
```

Output

Technical plan generated.

Artifacts:

- design/technical-design.md
- design/api-design.md
- design/db-design.md
- design/service-flow.md
- design/risk-analysis.md
- design/implementation-plan.md

Next:

aiteam tasks --run run_20260622_143000

22. Command: aiteam tasks

Purpose

Generate task breakdown.

Syntax

```
aiteam tasks [options]
```

Options

```
--run <runId>
```

Target run.

```
--format <format>
```

Output format.

Supported values:

```
markdown  
json
```

```
jira  
all
```

Default: `all`.

```
--regenerate
```

Regenerate task breakdown.

Examples

```
aiteam tasks --run run_20260622_143000
```

```
aiteam tasks --run run_20260622_143000 --format jira
```

Generated Artifacts

```
tasks/  
  task-breakdown.md  
  task-breakdown.json  
  jira-ready-tasks.md
```

Output

Task breakdown generated.

Epic:

- Invoice CSV Export

Stories:

- Add export invoice API
- Add filtering logic
- Generate CSV response
- Add tests

Artifacts:

- tasks/task-breakdown.md
- tasks/task-breakdown.json
- tasks/jira-ready-tasks.md

Next:

```
aiteam tests --run run_20260622_143000
```

23. Command: `aiteam tests`

Purpose

Generate test matrix and manual test checklist.

This command plans tests but does not execute tests.

Syntax

```
aiteam tests [options]
```

Options

```
--run <runId>
```

Target run.

```
--type <type>
```

Test planning type.

Supported values:

```
unit  
integration  
manual  
all
```

Default: `all`.

```
--regenerate
```

Regenerate test matrix.

Examples

```
aiteam tests --run run_20260622_143000
```

```
aiteam tests --run run_20260622_143000 --type unit
```

Generated Artifacts

```
tests/  
  test-matrix.md  
  test-matrix.json  
  manual-test-checklist.md
```

Output

```
Test matrix generated.  
  
Test cases:  
- TC-001 Export invoices with valid filters  
- TC-002 Reject invalid date range  
- TC-003 Return CSV header when no data  
- TC-004 Reject unauthorized user  
  
Artifacts:  
- tests/test-matrix.md  
- tests/test-matrix.json  
- tests/manual-test-checklist.md  
  
Next:  
aiteam report --run run_20260622_143000
```

24. Command: `aiteam code`

Purpose

Run a selected AI coding CLI to implement the approved plan.

This command should only run after the requirement, design, task breakdown, and test matrix are ready.

Syntax

```
aiteam code [options]
```

Options

```
--run <runId>
```

Target run.

```
--agent <agent>
```

Selected AI coding agent.

Supported values:

```
claude  
codex  
gemini  
aider
```

```
--approve
```

Approve code generation if approval is required.

```
--dry-run
```

Generate implementation prompt but do not run AI CLI.

```
--timeout <seconds>
```

Command timeout.

```
--max-iterations <number>
```

Maximum fix iterations.

```
--prompt-only
```

Only generate implementation prompt.

Examples

```
aiteam code --run run_20260622_143000 --agent codex
```

```
aiteam code --run run_20260622_143000 --agent claude --approve
```

```
aiteam code --run run_20260622_143000 --prompt-only
```

Actions

The command should:

1. Validate run has required artifacts.
2. Generate implementation prompt if missing.
3. Ask for approval unless already approved.
4. Save Git snapshot.
5. Run selected AI CLI.
6. Stream logs.
7. Enforce timeout.
8. Collect changed files.
9. Generate diff patch.
10. Detect protected file modifications.
11. Save implementation artifacts.

Generated Artifacts

```
implementation/  
  implementation-prompt.md  
  coding-checklist.md  
  agent-output.log  
  changed-files.json  
  diff.patch  
  implementation-notes.md
```

Output

Code generation completed.

Agent: codex

Changed files:

- src/main/java/com/example/invoice/InvoiceExportController.java
- src/main/java/com/example/invoice/InvoiceExportService.java
- src/test/java/com/example/invoice/InvoiceExportServiceTest.java

Artifacts:

- implementation/diff.patch
- implementation/changed-files.json
- implementation/agent-output.log

Next:

```
aiteam test --run run_20260622_143000
```

Error Cases

Approval required

```
Approval required before code generation.  
Use --approve to continue, or approve from local web UI.
```

Exit code: 15

Missing AI CLI

```
Error: Agent "codex" is not available.  
Run: aiteam doctor
```

Exit code: 5

Protected file modified

```
Warning: Protected file modified:  
- .env  
  
Workflow stopped.  
Review changes before continuing.
```

Exit code: 11

25. Command: `aiteam test`

Purpose

Run configured test/build commands.

This command executes tests. It is different from `aiteam tests`, which generates test plans.

Syntax

```
aiteam test [options]
```

Options

```
--run <runId>
```

Target run.

```
--build
```

Run build command.

```
--unit
```

Run unit test command.

```
--integration
```

Run integration test command.

```
--lint
```

Run lint command.

```
--all
```

Run all configured commands.

```
--command <command>
```

Run custom test command.

```
--timeout <seconds>
```

Command timeout.

Examples

```
aiteam test --run run_20260622_143000
```

```
aiteam test --run run_20260622_143000 --all
```

```
aiteam test --run run_20260622_143000 --command "mvn test"
```

Actions

The command should:

1. Load configured commands.
2. Run selected commands.
3. Capture stdout/stderr.
4. Save command logs.
5. Detect pass/fail.
6. Generate test result summary.

Generated Artifacts

```
tests/  
  test-result.md  
  failed-tests.md  
  
logs/  
  test.log
```

Output

```
Test execution completed.  
  
Build: PASSED  
Unit tests: FAILED  
Integration tests: SKIPPED  
Lint: SKIPPED  
  
Failed tests:  
- InvoiceExportServiceTest.shouldRejectInvalidDateRange  
  
Artifacts:  
- tests/test-result.md  
- tests/failed-tests.md  
- logs/test.log  
  
Next:  
aiteam review --run run_20260622_143000
```

Exit Behavior

If tests fail, the command should return exit code `8`, but still generate artifacts.

26. Command: `aiteam review`

Purpose

Review generated code, tests, and requirement coverage.

Syntax

```
aiteam review [options]
```

Options

```
--run <runId>
```

Target run.

```
--security
```

Run security review.

```
--coverage
```

Run requirement coverage review.

```
--all
```

Run all review types.

```
--regenerate
```

Regenerate existing review.

Examples

```
aiteam review --run run_20260622_143000
```

```
aiteam review --run run_20260622_143000 --all
```

Actions

The command should review:

- clarified requirement
- acceptance criteria
- task breakdown
- diff patch
- changed files
- test result

Generated Artifacts

```
review/  
  review-report.md  
  security-review.md  
  requirement-coverage.md
```

Output

```
Review completed.  
  
Status: CHANGES_REQUIRED  
  
Findings:  
- Missing test for empty CSV result.  
- Date range validation should reject fromDate after toDate.  
- No security issue found.  
  
Artifacts:  
- review/review-report.md  
- review/security-review.md  
- review/requirement-coverage.md  
  
Next:  
aiteam code --run run_20260622_143000 --agent codex  
or:  
aiteam report --run run_20260622_143000
```

27. Command: `aiteam trace`

Purpose

Generate or update traceability matrix.

Syntax

```
aiteam trace [options]
```

Options

```
--run <runId>
```

Target run.

```
--regenerate
```

Regenerate traceability.

```
--format <format>
```

Output format.

Supported values:

```
markdown  
json  
all
```

Default: `all`.

Examples

```
aiteam trace --run run_20260622_143000
```

```
aiteam trace --run run_20260622_143000 --format markdown
```

Generated Artifacts

```
report/  
  traceability.md  
  traceability.json
```

Output

```
Traceability generated.
```

```
Coverage:
```

- Covered: 6
- Partial: 1
- Not covered: 0

```
Artifacts:
```

- report/traceability.md
- report/traceability.json

28. Command: `aiteam report`

Purpose

Generate final delivery report.

Syntax

```
aiteam report [options]
```

Options

```
--run <runId>
```

Target run.

```
--regenerate
```

Regenerate report.

```
--include-logs
```

Include summarized logs.

```
--format <format>
```

Output format.

Supported values:

```
markdown
json
html
all
```

Default: `markdown`.

Examples

```
aiteam report --run run_20260622_143000
```

```
aiteam report --run run_20260622_143000 --format all
```

Generated Artifacts

```
report/
  final-report.md
  final-report.json
  final-report.html
```

Output

```
Final report generated.
```

```
Summary:
```

- Requirement clarified
- Technical design generated
- Task breakdown generated
- Test matrix generated
- Code generation skipped
- Review skipped

```
Artifacts:
```

- report/final-report.md

```
Run status: REPORT_GENERATED
```

29. Command: `aiteam artifacts`

Purpose

List artifacts generated for a run.

Syntax

```
aiteam artifacts <runId> [options]
```

Options

```
--type <type>
```

Filter by artifact type.

```
--json
```

Output JSON.

Examples

```
aiteam artifacts run_20260622_143000
```

```
aiteam artifacts run_20260622_143000 --type design
```

Output

```
Artifacts for run_20260622_143000

Requirement
- requirement/clarified-requirement.md
- requirement/acceptance-criteria.md

Design
- design/technical-design.md
- design/api-design.md

Tasks
- tasks/task-breakdown.md
```

```
Report
- report/final-report.md
```

30. Command: `aiteam open`

Purpose

Open generated artifacts or local UI.

Syntax

```
aiteam open [target] [options]
```

Targets

```
ui
run
report
diff
logs
config
```

Examples

```
aiteam open ui
```

```
aiteam open report --run run_20260622_143000
```

```
aiteam open diff --run run_20260622_143000
```

```
aiteam open config
```

Output

```
Opening report:
.ai-team/runs/run_20260622_143000/report/final-report.md
```

31. Command: `aiteam export`

Purpose

Export generated artifacts.

Syntax

```
aiteam export <runId> [options]
```

Options

```
--format <format>
```

Export format.

Supported values:

```
markdown  
zip  
json  
html
```

```
--output <path>
```

Output path.

```
--include-logs
```

Include logs.

```
--include-diff
```

Include diff patch.

Examples

```
aiteam export run_20260622_143000 --format zip
```

```
aiteam export run_20260622_143000 --format markdown --output ./delivery-docs
```

Output

Export completed.

Output:
./delivery-docs/run_20260622_143000.zip

32. Command: `aiteam clean`

Purpose

Clean old runs, logs, or temporary files.

Syntax

```
aiteam clean [options]
```

Options

```
--runs
```

Clean old runs.

```
--logs
```

Clean logs.

```
--older-than <duration>
```

Clean items older than duration.

Examples:

```
7d  
30d  
90d
```

```
--dry-run
```

Show what would be deleted.

```
--yes
```

Confirm deletion.

Examples

```
aiteam clean --logs --older-than 30d
```

```
aiteam clean --runs --older-than 90d --dry-run
```

Safety

This command must ask for confirmation unless `--yes` is provided.

33. Command: `aiteam prompts`

Purpose

Manage prompt templates.

Syntax

```
aiteam prompts <subcommand> [options]
```

Subcommands

```
aiteam prompts list  
aiteam prompts show <name>  
aiteam prompts edit <name>  
aiteam prompts reset <name>  
aiteam prompts validate
```

Examples

```
aiteam prompts list
```

```
aiteam prompts show ba-agent
```

```
aiteam prompts edit architect-agent
```

```
aiteam prompts reset developer-agent
```

Output

```
Prompt templates:
```

- ba-agent.md
- po-agent.md
- pm-agent.md
- architect-agent.md
- developer-agent.md
- qa-agent.md
- reviewer-agent.md
- security-reviewer-agent.md
- reporter-agent.md

34. Command: `aiteam templates`

Purpose

Manage project templates.

Syntax

```
aiteam templates <subcommand> [options]
```

Subcommands

```
aiteam templates list  
aiteam templates show <name>  
aiteam templates apply <name>
```

Examples

```
aiteam templates list
```

```
aiteam templates apply java-spring-boot
```

Output

Available templates:

- java-spring-boot
- node-nestjs
- react-vite
- generic

35. Command: `aiteam github`

Purpose

Manage GitHub integration.

GitHub integration should be optional.

Syntax

```
aiteam github <subcommand> [options]
```

Subcommands

```
aiteam github status  
aiteam github test  
aiteam github pr create
```

```
aiteam github pr view  
aiteam github actions
```

Examples

```
aiteam github status
```

```
aiteam github pr create --run run_20260622_143000
```

```
aiteam github actions --run run_20260622_143000
```

MVP Recommendation

Use GitHub CLI first.

The product can call:

```
gh auth status  
gh pr create  
gh pr view  
gh run list
```

Approval

Creating a PR should require approval unless user explicitly passes:

```
--approve
```

36. Command: `aiteam jira`

Purpose

Manage Jira integration.

Jira integration should be optional.

Syntax

```
aiteam jira <subcommand> [options]
```

Subcommands

```
aiteam jira status  
aiteam jira test  
aiteam jira export  
aiteam jira create  
aiteam jira comment
```

Examples

```
aiteam jira status
```

```
aiteam jira export --run run_20260622_143000
```

```
aiteam jira create --run run_20260622_143000
```

```
aiteam jira comment --run run_20260622_143000 --issue ABC-123
```

MVP Behavior

MVP should support:

```
aiteam jira export --run <runId>
```

This generates Jira-ready markdown without calling Jira API.

Future Behavior

Later, the CLI can create:

- epic
- story
- subtask
- comment
- task link

Approval

API-based Jira updates must require approval.

37. Command: `aiteam slack`

Purpose

Manage Slack integration.

Slack integration should be optional.

Syntax

```
aiteam slack <subcommand> [options]
```

Subcommands

```
aiteam slack status  
aiteam slack test  
aiteam slack post
```

Examples

```
aiteam slack status
```

```
aiteam slack test
```

```
aiteam slack post --run run_20260622_143000
```

MVP Behavior

MVP should support post-only behavior:

- post final report summary
- post PR-ready notification
- post workflow result

Receiving Slack events is out of scope for MVP.

Approval

Posting to Slack must require approval unless user passes:

```
--approve
```

38. Command: `aiteam rollback`

Purpose

Rollback code changes generated during a run.

This command is dangerous and must require confirmation.

Syntax

```
aiteam rollback <runId> [options]
```

Options

```
--dry-run
```

Show what would be reverted.

```
--yes
```

Confirm rollback.

Examples

```
aiteam rollback run_20260622_143000 --dry-run
```

```
aiteam rollback run_20260622_143000 --yes
```

Actions

The command should:

1. Check changed files for the run.

2. Show files that will be reverted.
3. Ask for confirmation.
4. Revert changes if approved.
5. Save rollback log.

Output

Rollback preview:

Files to revert:

- src/main/java/com/example/auth/PasswordResetService.java
- src/test/java/com/example/auth/PasswordResetServiceTest.java

Run with --yes to confirm.

39. Command: `aiteam approve`

Purpose

Approve a pending workflow gate from CLI.

Syntax

```
aiteam approve <runId> [options]
```

Options

```
--gate <gate>
```

Gate to approve.

Supported values:

```
requirement  
plan  
code  
external-update  
rollback
```

```
--note <note>
```

Approval note.

Examples

```
aiteam approve run_20260622_143000 --gate plan
```

```
aiteam approve run_20260622_143000 --gate code --note "Plan looks safe"
```

Output

```
Approval recorded.
```

```
Run: run_20260622_143000
```

```
Gate: code
```

```
Note: Plan looks safe
```

40. Command: `aiteam reject`

Purpose

Reject a pending workflow gate from CLI.

Syntax

```
aiteam reject <runId> [options]
```

Options

```
--gate <gate>
```

Gate to reject.

```
--reason <reason>
```

Rejection reason.

Examples

```
aiteam reject run_20260622_143000 --gate plan --reason "API design is too complex"
```

Output

```
Gate rejected.
```

```
Run: run_20260622_143000
```

```
Gate: plan
```

```
Reason: API design is too complex
```

41. Command Naming Notes

41.1 tests vs test

The CLI should distinguish:

```
aiteam tests
```

Generates test matrix and test cases.

```
aiteam test
```

Executes configured test commands.

This distinction is important.

41.2 plan vs tasks

```
aiteam plan
```

Generates technical plan.

```
aiteam tasks
```

Generates task breakdown.

41.3 review vs report

```
aiteam review
```

Reviews code/test/coverage.

```
aiteam report
```

Summarizes final delivery.

42. Recommended MVP Commands

The MVP should include only the most important commands.

MVP Required

```
aiteam init  
aiteam ui  
aiteam doctor  
aiteam run  
aiteam new  
aiteam spec  
aiteam plan  
aiteam tasks  
aiteam tests  
aiteam report  
aiteam list  
aiteam show
```

MVP+ Commands

```
aiteam code  
aiteam test  
aiteam review  
aiteam trace  
aiteam artifacts  
aiteam open  
aiteam prompts
```

Later Commands

```
aiteam github  
aiteam jira  
aiteam slack  
aiteam rollback  
aiteam approve  
aiteam reject  
aiteam export  
aiteam clean
```

43. Example Full CLI Session

Step 1: Initialize project

```
cd hotel-booking-service  
aiteam init --type java-spring-boot
```

Step 2: Check environment

```
aiteam doctor
```

Step 3: Start local UI

```
aiteam ui --open
```

Step 4: Create docs-only run

```
aiteam run "Thêm API export invoice CSV theo hotelId, date range, status" --  
mode docs-only
```

Step 5: View run

```
aiteam list  
aiteam show run_20260622_143000 --artifacts
```

Step 6: Generate implementation prompt

```
aiteam run "Thêm API export invoice CSV theo hotelId, date range, status" --mode assisted
```

Step 7: Run coding agent

```
aiteam code --run run_20260622_143000 --agent codex
```

Step 8: Run tests

```
aiteam test --run run_20260622_143000 --all
```

Step 9: Review

```
aiteam review --run run_20260622_143000 --all
```

Step 10: Generate final report

```
aiteam trace --run run_20260622_143000  
aiteam report --run run_20260622_143000
```

44. Final CLI Summary

The CLI should support the full local software delivery workflow:

```
init project  
→ create run  
→ clarify requirement  
→ define scope  
→ analyze repo  
→ create design  
→ create tasks  
→ create test matrix  
→ optionally run AI coding agent  
→ run tests  
→ review changes  
→ generate traceability  
→ generate final report
```

The most important commands for the first version are:

```
aiteam init  
aiteam ui  
aiteam doctor  
aiteam run  
aiteam spec  
aiteam plan  
aiteam tasks  
aiteam tests  
aiteam report
```

The product should start with documentation and workflow commands first.

Code execution, integrations, rollback, and PR creation should come after the core workflow is stable.